ÉTUDE DE CAS — STRIPE

LIVRABLE 1

Diagramme complet de l’architecture des données

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l’infrastructure de données et de calcul pour l’IA

1. Schéma de l'architecture de données

Le diagramme ci-dessous présente l'architecture de données proposée pour Stripe. Il montre comment les systèmes OLTP, OLAP et NoSQL sont reliés entre eux, comment les données circulent de l'un à l'autre et à quel moment le modèle de détection de fraude intervient. L'infrastructure complète utilise des services gérés par AWS (Amazon).

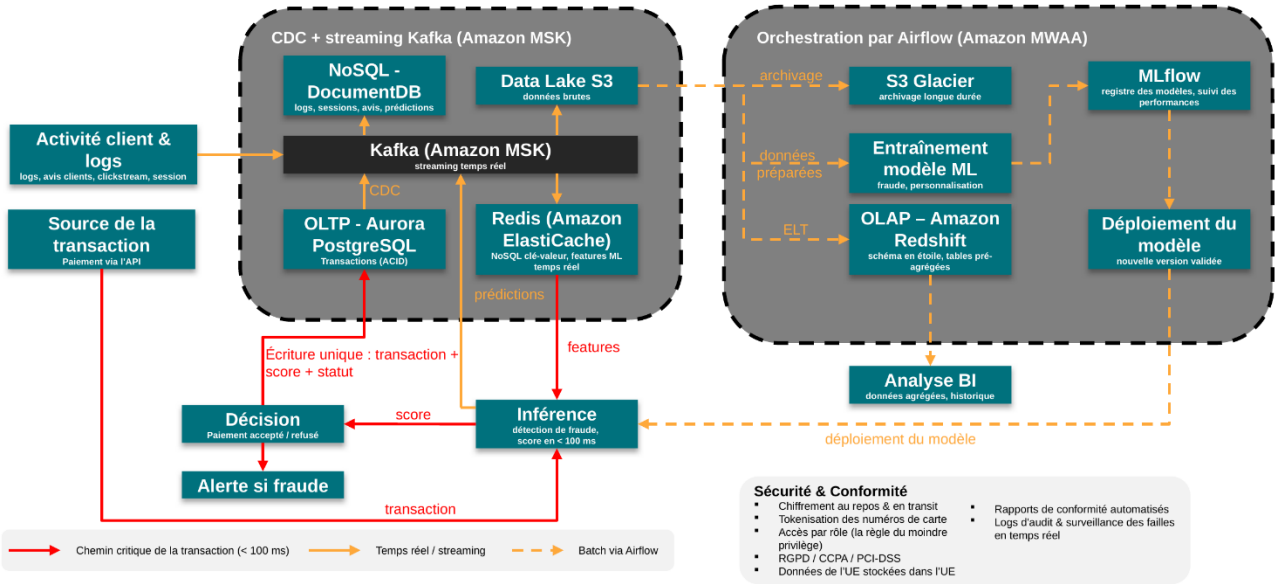


Figure 1 – Architecture des données de Stripe sur AWS

2. Parcours de la donnée

Pour faciliter la lecture du schéma, le parcours d'une transaction est décrit ci-dessous étape par étape.

1. **Réception de la transaction.** Lorsqu'un client paie chez un marchand, la transaction arrive par l'API de paiement. Elle est envoyée directement au service d'inférence, avant tout enregistrement.
2. **Détection de fraude en temps réel.** Le service d'inférence récupère depuis Redis les features déjà calculées pour ce client et ce marchand, comme le nombre de paiements dans la dernière heure ou le montant moyen des 30 derniers jours. Il les combine avec les informations de la transaction (montant, pays, appareil…) et calcule un score de fraude. Cette étape doit se faire en moins de 100 ms afin de ne pas ralentir le paiement.
3. **Décision et enregistrement.** En fonction du score, le paiement est accepté ou refusé. La transaction est alors enregistrée une seule fois dans la base OLTP Aurora PostgreSQL, avec son score et son statut. Si le paiement est suspect, une alerte est envoyée à l'équipe fraude. Le service d'inférence publie aussi sa prédiction (score, décision, version du modèle) dans Kafka, afin qu'elle soit conservée dans DocumentDB pour le suivi du modèle. Cet envoi se fait en parallèle et ne ralentit donc pas le paiement.
4. **Capture des changements.** Chaque nouvelle ligne enregistrée dans l'OLTP est capturée par le CDC (Change Data Capture) et envoyée dans Kafka. Les données d'activité client (logs, avis clients, clickstream, sessions) arrivent également dans Kafka, directement depuis le site web et les applications.
5. **Distribution des données.** Kafka transmet ensuite les données vers trois destinations :
    a. DocumentDB, qui stocke les logs, les sessions, les avis clients et les prédictions du modèle.
    b. Le Data Lake S3, qui conserve toutes les données brutes.
    c. Redis, dont les features sont recalculées en continu à partir des nouveaux événements. La prochaine transaction du même client sera donc scorée avec les features mises à jour.
6. **Traitements batch.** Plusieurs traitements sont planifiés par Airflow à partir du Data Lake :
    a. Chaque mois, les données anciennes sont archivées dans S3 Glacier pour une conservation longue durée à faible coût.

   b. Toutes les heures, un ELT charge les nouvelles données dans le data warehouse Redshift puis les transforme en SQL. Redshift est organisé en schéma en étoile avec des tables pré-agrégées. Les équipes peuvent ainsi réaliser leurs analyses BI (revenus, segmentation client, fraude).
   c. Les données sont préparées puis utilisées pour entraîner chaque semaine le modèle de Machine Learning (ML).

7. **Mise à jour du modèle.** Chaque nouveau modèle entraîné est enregistré dans MLflow, qui conserve ses versions et suit ses performances. Une fois validé, il est déployé dans le service d'inférence et remplace l'ancienne version.
8. **Sécurité et conformité.** Ces mesures s'appliquent à toute l'architecture : chiffrement des données, tokenisation des numéros de carte, accès par rôle, logs d'audit, surveillance des failles en temps réel et rapports de conformité automatisés. Elles permettent de répondre au RGPD, à la CCPA et à PCI-DSS. Le détail est présenté dans le livrable 6.

3. Les services utilisés

Comme précisé précédemment, l'infrastructure est entièrement déployée sur AWS. Elle s'appuie principalement sur des services gérés par AWS, complétés par quelques outils open source (Debezium, MLflow) :

Composant	Rôle	Service
OLTP	Enregistrement des transactions (ACID), bascule automatique en cas de panne	Amazon Aurora PostgreSQL
Streaming et CDC	Capture des changements de l'OLTP et transport des données en temps réel	Kafka (Amazon MSK, Managed Streaming for Apache Kafka) + Debezium
NoSQL documents	Stockage des logs, sessions, avis clients et prédictions du modèle	Amazon DocumentDB
Features temps réel	Mise à disposition des features ML en quelques millisecondes	Redis (Amazon ElastiCache)
Data Lake et archivage	Stockage des données brutes, puis archivage longue durée	Amazon S3 · S3 Glacier
Orchestration	Planification des traitements batch (archivage, ELT, entraînement)	Airflow (Amazon MWAA, Managed Workflows for Apache Airflow)
OLAP	Data warehouse pour les analyses BI sur l'historique	Amazon Redshift
Machine learning	Entraînement, registre des modèles, inférence en temps réel	MLflow · API FastAPI (Amazon ECS, Elastic Container Service) · instances EC2 pour l'entraînement
Sécurité et conformité	Chiffrement, accès par rôle, audit (détail au livrable 6)	AWS KMS (Key Management Service) · IAM · CloudTrail
Supervision et coûts	Suivi des performances, alertes, suivi du budget	Amazon CloudWatch · AWS Cost Explorer · AWS Budgets
Déploiement de l'infrastructure	Infrastructure décrite en code, versionnée dans Git	Terraform

4. Choix d'hébergement

L'infrastructure est hébergée dans le cloud public AWS. Une solution sur site (on-premise) demanderait un gros investissement de départ et des serveurs dimensionnés pour les pics, donc souvent sous-utilisés. De plus il faudrait prendre en compte la maintenance pour une solution on-premise à gérer par Stripe. Une solution hybride obligerait à maintenir deux infrastructures, avec des échanges plus lents entre les deux, ce qui est difficile à concilier avec les 100 ms du paiement. Le cloud permet aussi de garder les données des clients européens dans une région de l'UE, comme le demande le RGPD (livrable 6).

Type de service	Utilisé pour	Pourquoi
PaaS (services managés)	Aurora, DocumentDB, ElastiCache, MSK, Redshift, MWAA	AWS gère les correctifs, les sauvegardes et la réplication

Type de service	Utilisé pour	Pourquoi
Serverless (conteneurs)	Inférence, calcul des features, MLflow (ECS sur Fargate)	Aucun serveur à gérer, nombre de conteneurs ajusté à la charge
IaaS (machines virtuelles)	Entraînement des modèles (EC2)	Choix précis de la machine, supprimée après l'entraînement

5. Architecture physique

Le schéma de la section 1 montre les composants et les flux (architecture logique). Celui-ci montre où ils sont installés physiquement.

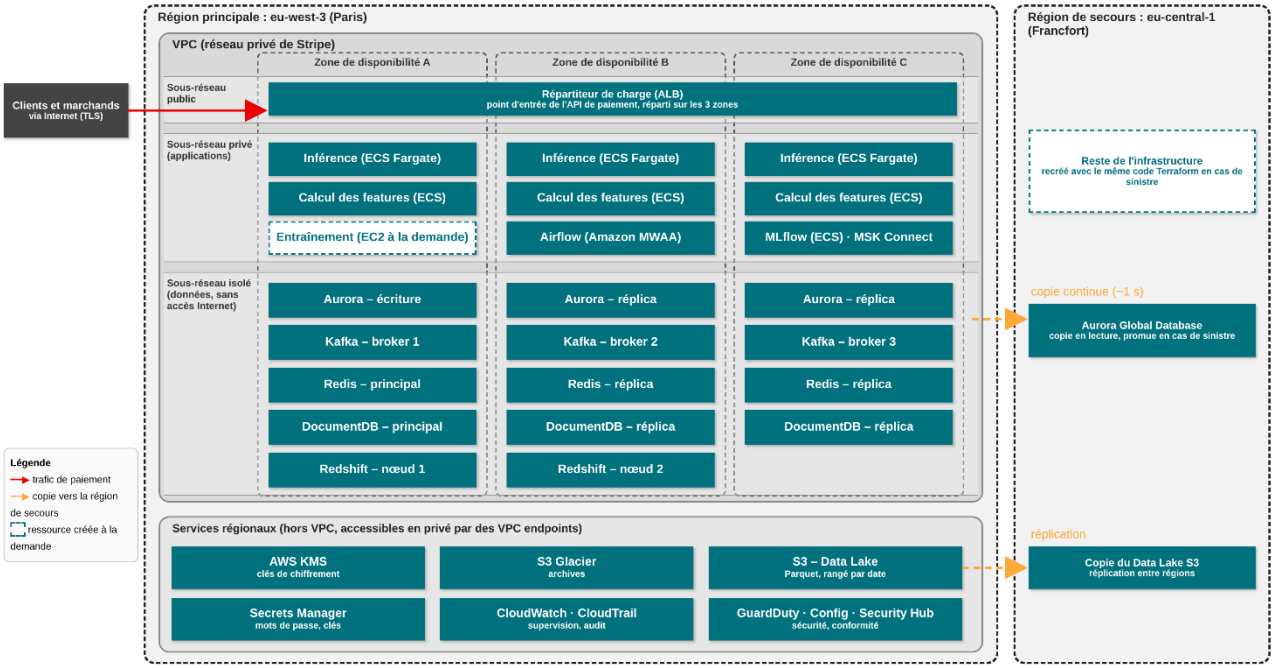


Figure 2 – Architecture physique : régions, zones de disponibilité et réseau

- **Région et zones :** l'infrastructure est déployée dans la région AWS de Paris. Chaque composant est réparti sur trois zones de disponibilité (trois centres de données séparés) : la panne de l'un d'eux n'arrête pas le service.
- **Région de secours :** la région de Francfort reçoit en continu une copie de la base OLTP et du Data Lake, en cas de sinistre sur toute la région de Paris.
- **Réseau :** seul le point d'entrée de l'API est accessible depuis Internet. Les applications sont dans un sous-réseau privé et les bases de données dans un sous-réseau isolé, sans accès à Internet.

6. Dimensionnement

Le sujet ne donne pas de volumes. Les hypothèses retenues pour ce dossier sont : 10 millions de transactions par jour, avec des pics à 1 200 par seconde, 50 millions de clients et 10 millions de sessions de navigation par jour.

Composant	Ressources prévues	Justification
Inférence	Conteneurs de 2 vCPU et 4 Go, de 3 à 12 selon la charge	Une prédiction XGBoost prend quelques millisecondes sur CPU. Pas de GPU, inutile pour des prédictions faites une par une
Entraînement	Une machine de 32 vCPU et 64 Go, environ 3 h par semaine	XGBoost s'entraîne bien sur CPU. Un GPU peut être ajouté si l'entraînement devient trop long
Aurora (OLTP)	1 instance d'écriture et 2 réplicas, environ 7 To (24 mois)	Environ 10 Go de transactions par jour
DocumentDB	3 instances, environ 12 To	Sessions et prédictions sur 13 mois, logs sur 90 jours

Composant	Ressources prévues	Justification
Redis	3 nœuds, environ 10 Go en mémoire	Features de tous les clients et marchands, lues en moins d'1 ms
Data Lake S3 et Redshift	Environ 10 To par an dans S3, 2 To dans Redshift pour 5 ans	Données compressées (Parquet, stockage en colonnes)

Avec Redis, le modèle et l'écriture dans Aurora, une transaction est traitée en une vingtaine de millisecondes, bien en dessous des 100 ms demandés.

7. Scalabilité et haute disponibilité

- **Scalabilité :** les conteneurs sont ajoutés automatiquement quand la charge augmente (auto-scaling). Kafka est découpé en partitions, Redshift répartit les données sur plusieurs nœuds, et Aurora comme DocumentDB peuvent être fragmentées (sharding) sur plusieurs machines si une seule ne suffit plus. Les index, le cache Redis et les tables pré-agrégées accélèrent les lectures (livrables 2, 3 et 4).

Les principaux scénarios de panne ont été anticipés :

Panne	Réaction prévue	Impact
Base Aurora principale	Un réplica d'une autre zone prend le relais automatiquement (failover)	Environ 30 secondes sans écriture, aucune transaction perdue
Un centre de données	Les services continuent dans les deux autres zones	Pas d'interruption visible
Service d'inférence	Règles simples de secours, transaction marquée « non scorée » (livrable 7)	Le paiement continue
Toute la région de Paris	La copie de Francfort devient la base principale, le reste est recréé avec Terraform	Reprise en quelques minutes

8. Déploiement et supervision

- **Infrastructure as Code :** toute l'infrastructure est décrite en code avec Terraform et stockée dans Git. Elle peut ainsi être recréée à l'identique, par exemple dans la région de secours, et chaque modification est vérifiée avant d'être appliquée.
- **Supervision :** Amazon CloudWatch suit la latence de l'inférence, les erreurs, le retard de Kafka et l'utilisation des machines, et envoie une alerte en cas de problème.

9. Coûts de l'infrastructure

Le tableau donne le coût mensuel estimé à partir des prix publics d'AWS pour la région de Paris (tarifs à la demande, octobre 2026) et des hypothèses de la section 6. Les postes facturés à l'usage (opérations de lecture et d'écriture des bases, volume de journaux analysés) dépendent du trafic réel : leur montant reste une estimation.

Poste	Coût mensuel (base du calcul)	Comment le réduire
DocumentDB	5 200 $: 3 instances db.r6g.2xlarge à 1,216 $/h, 12 To à 0,116 $/Go, opérations d'E/S	Durées de conservation courtes (TTL), données anciennes uniquement dans S3, instances réservées
Aurora PostgreSQL	4 500 $: 3 instances db.r6g.2xlarge à 1,196 $/h, 7 To à 0,11 $/Go, opérations d'E/S	Instances réservées (0,786 $/h au lieu de 1,196 $/h sur 1 an), transactions de plus de 24 mois retirées de la base
Région de secours	2 800 $: une instance Aurora, copie de la base et copie du Data Lake	Seules la base et le Data Lake y sont copiés, le reste n'est créé qu'en cas de sinistre
Redshift	1 900 $: 2 nœuds ra3.xlplus à 1,26 $/h, 2 To à 0,025 $/Go	Instances réservées, requêtes sur les tables pré-agrégées
Sécurité et supervision	Environ 1 900 $: GuardDuty, AWS Config, CloudWatch, Security Hub, KMS (selon le volume de journaux)	Durée de conservation des logs limitée, filtrage des journaux analysés

Poste	Coût mensuel (base du calcul)	Comment le réduire
Kafka (MSK)	1 650 $: 3 brokers m7g.xlarge à 0,476 $/h, 2 To à 0,116 $/Go, MSK Connect à 0,128 $/h par unité	Messages conservés 7 jours seulement (tout est copié dans S3)
Redis (ElastiCache)	1 050 $: 3 nœuds cache.r6g.xlarge à 0,479 $/h	Durée de vie des clés (TTL), passage au moteur Valkey compatible Redis (0,383 $/h, environ -20 %)
Data Lake S3 et Glacier	700 $: 20 To en S3 Standard à 0,024 $/Go, archives Glacier Deep Archive à 0,0018 $/Go	Passage automatique vers Glacier, format Parquet compressé
Airflow (MWAA)	650 $: environnement Medium à 0,888 $/h	Taille d'environnement adaptée au nombre de DAG
Conteneurs (Fargate)	520 $: une dizaine de conteneurs en moyenne, processeurs Graviton (0,039 $ par vCPU et 0,0042 $ par Go de mémoire, par heure)	Auto-scaling : moins de conteneurs la nuit
Réseau	420 $: répartiteur de charge, passerelles NAT, échanges entre zones	Points d'accès privés vers S3 (VPC endpoints)
Entraînement des modèles	20 $: une instance c7g.8xlarge à 1,37 $/h, environ 13 h par mois	Machines supprimées après l'entraînement, instances Spot
Total	Environ 21 000 $ par mois	Environ 18 000 $ avec des réservations sur 1 an

Les plus gros postes sont les bases de données qui tournent en permanence (DocumentDB, Aurora et sa copie de secours, Redshift). C'est donc sur elles que portent les principales économies : une réservation sur 1 an réduit le prix d'une instance d'environ un tiers (par exemple 0,786 $/h au lieu de 1,196 $/h pour une instance Aurora). Ce chiffrage peut être retrouvé et ajusté avec AWS Pricing Calculator.

- **Suivi du budget (FinOps) :** AWS Budgets envoie une alerte si le coût du mois dépasse le budget prévu, ce qui donne à la direction financière un coût mensuel prévisible.
- **Consommation d'énergie (GreenOps) :** les machines d'entraînement sont arrêtées dès la fin de l'entraînement, le GPU n'est pas utilisé quand il n'apporte rien, et les données inutiles sont supprimées automatiquement.